

♥ Heart Disease Prediction

A No-Code Machine Learning Workflow in Orange Data Mining

Orange Data Mining 3.x

1,025 patient records

Best model: Tree — AUC 0.986

Dataset: heart.csv | Process Report | July 2026

Contents

- 1. Project Overview
- 2. What is Orange Data Mining?
- 3. The Dataset
- 4. Workflow Architecture & Widget Connections
- 5. Step-by-Step Process
- 6. Exploratory Data Analysis
- 7. Model Training & Evaluation
- 8. Results
- 9. Key Takeaways
- 10. How to Reproduce

1. Project Overview

This project explores the classic **Heart Disease dataset** using **Orange Data Mining** — a free, open-source visual programming tool that lets you build data-mining and machine-learning pipelines by dragging and connecting widgets, with no code required. The goal: identify which clinical factors are most associated with heart disease, and compare multiple classification models to find the most reliable predictor.

Four models were trained and evaluated — **Tree**, **Naive Bayes**, **Logistic Regression**, and **kNN** — with the decision tree ultimately achieving the strongest performance (AUC 0.986).

2. What is Orange Data Mining?

Orange is built on top of familiar Python data-science libraries (including scikit-learn), but instead of writing code, the user drags **widgets** (icons) onto a canvas and connects them with lines. Each widget performs one job — loading data, drawing a chart, training a model, evaluating results — and data flows from widget to widget like a pipeline.

Think of each widget as a function: it takes data in on the left, does one specific job, and passes data out on the right. A connection line between two widgets is simply "the output of A becomes the input of B."

Widgets used in this project fall into three categories:

Category	Widgets	Purpose
Data	CSV File Import, Data Table, Edit Domain, Select Columns	Load and prepare the dataset
Visualize	Column Statistics, Distributions, Box Plot, Scatter Plot, Correlations	Explore patterns before modeling
Model / Evaluate	Tree, Naive Bayes, Logistic Regression, kNN, Test and Score, Confusion Matrix, ROC Analysis, Tree Viewer	Train and assess predictive models

3. The Dataset

data/heart.csv — 1,025 rows × 14 columns, no missing values.

#	Column	Description
1	age	Age in years
2	sex	1 = male, 0 = female
3	cp	Chest pain type (4 values)
4	trestbps	Resting blood pressure
5	chol	Serum cholesterol (mg/dl)
6	fbs	Fasting blood sugar > 120 mg/dl
7	restecg	Resting ECG results (0, 1, 2)
8	thalach	Maximum heart rate achieved
9	exang	Exercise-induced angina
10	oldpeak	ST depression induced by exercise
11	slope	Slope of peak exercise ST segment
12	ca	Major vessels (0–3) colored by fluoroscopy
13	thal	0 = normal, 1 = fixed defect, 2 = reversible defect
14	target	1 = heart disease present, 0 = absent

4. Workflow Architecture & Widget Connections

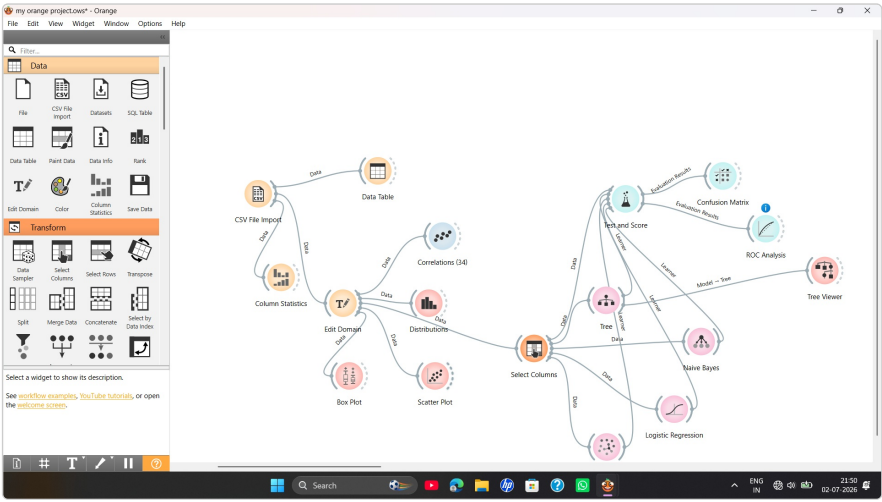


Fig. 1 — Full Orange canvas, all widgets and connections

```
+> Data Table
CSV File Import -----+
+> Edit Domain --+> Column Statistics
+> Distributions
+> Box Plot
+> Scatter Plot
+> Correlations

Edit Domain -> Select Columns -----+> Naive Bayes
+> Logistic Regression --+> Test and Score -> Confusion Matrix
+> kNN -----+> ROC Analysis

Tree -> Tree Viewer
```

There are three parallel "lanes" in this pipeline:

- **Load & clean:** CSV File Import → Edit Domain
- **Explore:** Edit Domain → five EDA widgets (all dead ends — for viewing only)
- **Model & evaluate:** Edit Domain → Select Columns → 4 models → Test and Score → Confusion Matrix / ROC / Tree Viewer

Why each connection exists

Connection	What it does	Why
CSV Import → everything	Reads heart.csv into memory	Entry point — nothing runs before this loads
CSV Import → Data Table	Displays raw rows	Sanity-check the load (1,025 rows, no odd values)
CSV Import → Edit Domain	Re-types cp, restecg, slope, ca, thal as categorical	These are coded labels, not true numeric quantities — treating "thal=3" as literally 3x "thal=1" would mislead a model
Edit Domain → 5 EDA widgets	Column Statistics, Distributions, Box Plot, Scatter Plot, Correlations	Each answers a different exploratory question on the <i>corrected</i> data; none feed further downstream
Edit Domain → Select Columns	Marks which columns are features vs. the target (target)	Models need to know what they're predicting
Select Columns → 4 models	Feeds the identical prepared dataset to Tree, Naive Bayes, Logistic Regression, kNN	Fair, parallel comparison — every model sees exactly the same inputs
4 models → Test and Score	Cross-validates all four learners together	Produces one shared comparison table (AUC, accuracy, F1, etc.)
Test and Score → Confusion Matrix	Breaks one model's results into hits/misses per class	Shows exactly how many patients were correctly/incorrectly classified
Test and Score → ROC Analysis	Plots true-positive vs. false-positive rate for all 4 models	Visual comparison of trade-offs at every decision threshold
Tree → Tree Viewer	Connects to the trained Tree model directly (not through Test and Score)	Tree Viewer needs the actual tree structure, not evaluation scores, to draw decision splits

5. Step-by-Step Process

1 Load the data

Imported `heart.csv` via **CSV File Import** — 1,025 instances, 14 features, no missing data.

2 Fix column types

Used **Edit Domain** to re-type `cp`, `restecg`, `slope`, `ca`, and `thal` as categorical rather than numeric.

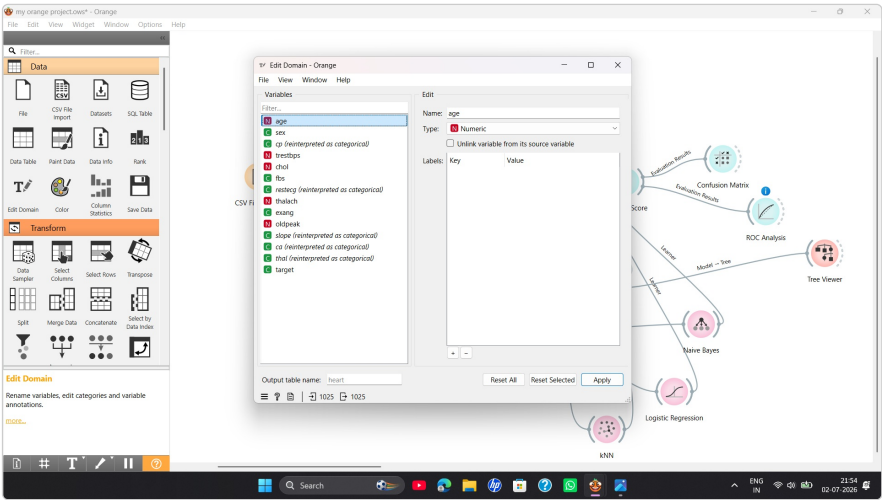


Fig. 2 — Edit Domain widget re-typing variables

3 Inspect the raw data

Fig. 3 — Data Table view of the 1,025 patient records

4 Explore feature statistics and distributions

Ran **Column Statistics**, **Distributions**, **Box Plot**, and **Scatter Plot** to understand the shape of every variable before modeling.

5 Select features and train four models in parallel

Connected the cleaned data through **Select Columns** into **Tree**, **Naive Bayes**, **Logistic Regression**, and **kNN**.

6 Evaluate with cross-validation

All four learners were fed into **Test and Score** (5-fold stratified cross-validation) to compare AUC, accuracy, F1, precision, recall, and MCC on equal footing.

7 Diagnose errors and interpret the winning model

Used **Confusion Matrix** and **ROC Analysis** to inspect performance, and **Tree Viewer** to see the actual decision logic of the best model.

6. Exploratory Data Analysis

6.1 Column Statistics

Summary statistics for every feature (mean, mode, median, dispersion, min/max).

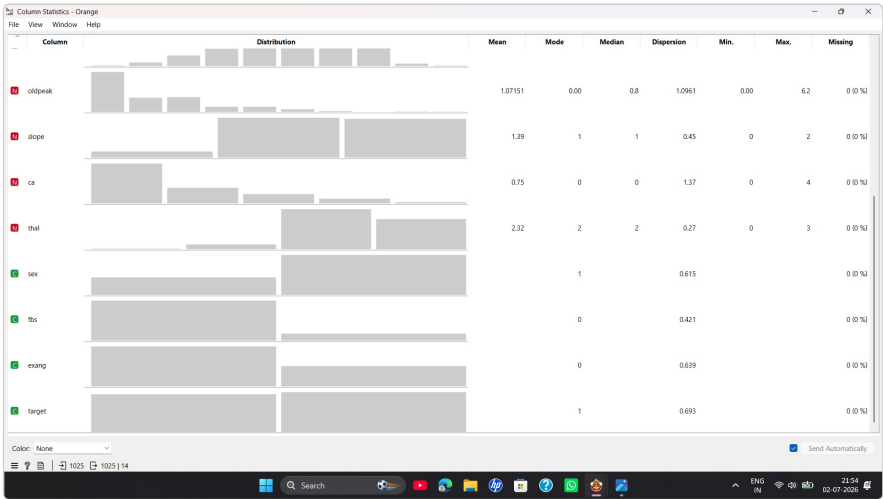


Fig. 4 — Column Statistics: mean, mode, median, dispersion for each variable

6.2 Age Distribution

Patients average **54.4 ± 9.1 years old** (median 56, interquartile range 48-61).

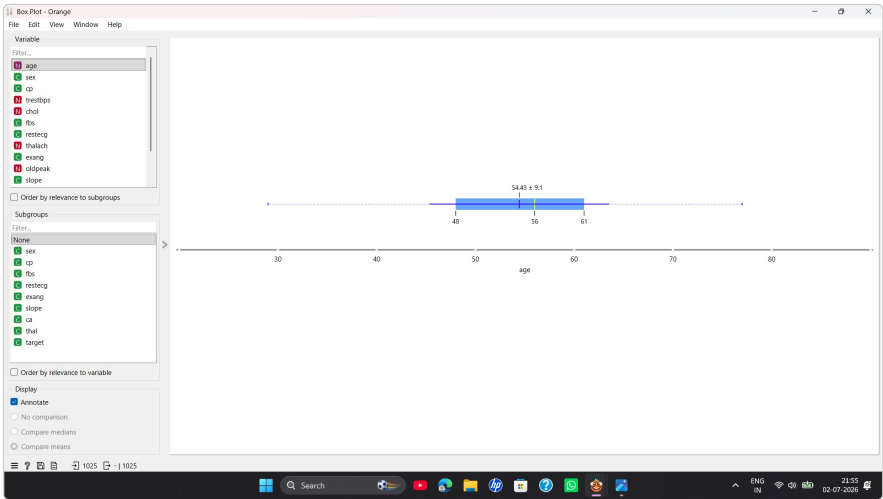


Fig. 5 — Box Plot of patient age

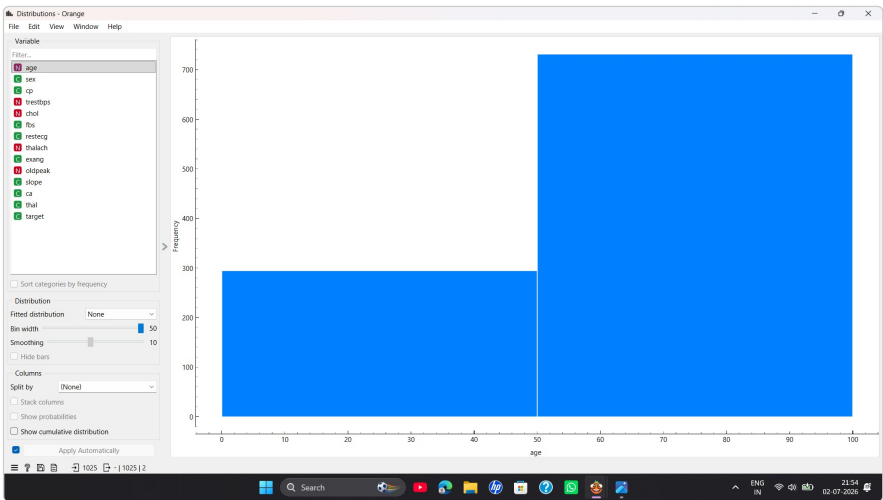


Fig. 6 — Age distribution histogram

6.3 Scatter Plot

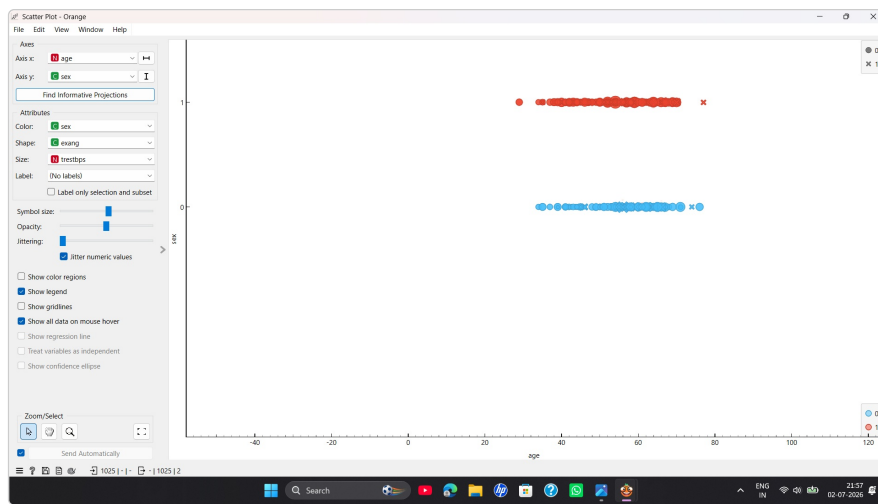


Fig. 7 — Age vs. sex, shaped by exercise-induced angina, sized by resting blood pressure

6.4 Feature Correlations (Pearson)

Rank	Features	Correlation
1	age : thalach	−0.390
2	oldpeak : thalach	−0.350
3	age : trestbps	+0.271
4	age : chol	+0.220
5	age : oldpeak	+0.208
6	oldpeak : trestbps	+0.187
7	chol : trestbps	+0.128
8	chol : oldpeak	+0.065
9	thalach : trestbps	−0.039
10	chol : thalach	−0.022

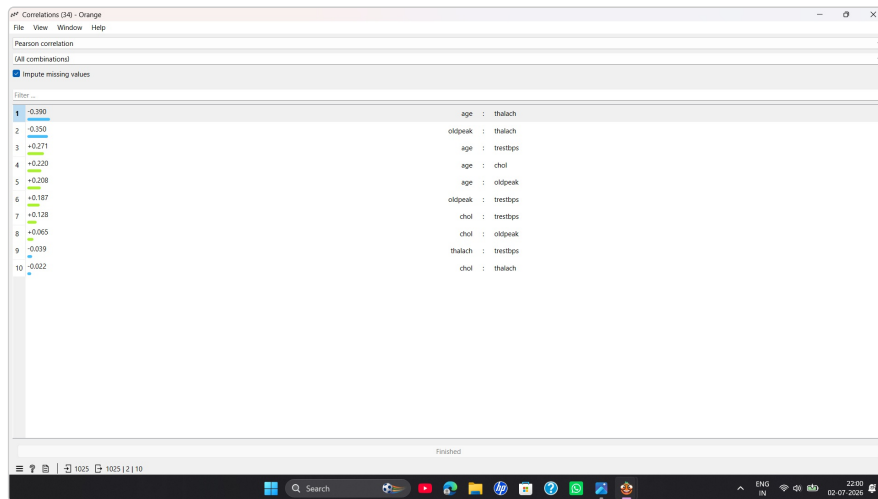


Fig. 8 — Pearson correlation ranking between numeric features

The strongest relationship is that **older patients tend to have a lower maximum heart rate** (age:thalach, $r = -0.39$) — a well-known physiological pattern. No feature pair is strongly correlated (all $|r| < 0.4$), so multicollinearity is not a major concern for the linear model.

7. Model Training & Evaluation

7.1 Cross-Validated Performance

5-fold stratified cross-validation results:

Model	AUC	CA (Accuracy)	F1	Precision	Recall	MCC
🌳 Tree	0.986	0.957	0.957	0.957	0.957	0.914
Logistic Regression	0.937	0.866	0.866	0.866	0.866	0.733
Naive Bayes	0.919	0.846	0.846	0.846	0.846	0.691
kNN	0.864	0.726	0.726	0.727	0.726	0.452

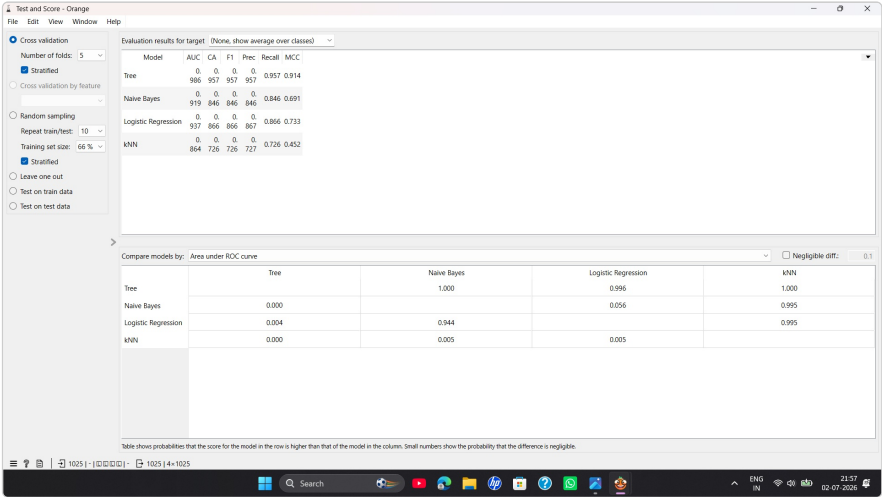


Fig. 9 — Test and Score comparison table and pairwise significance

The pairwise comparison confirms Tree's win is statistically robust: the probability that Tree scores higher than Naive Bayes or kNN is effectively 1.000, and 0.996 against Logistic Regression.

7.2 ROC Analysis

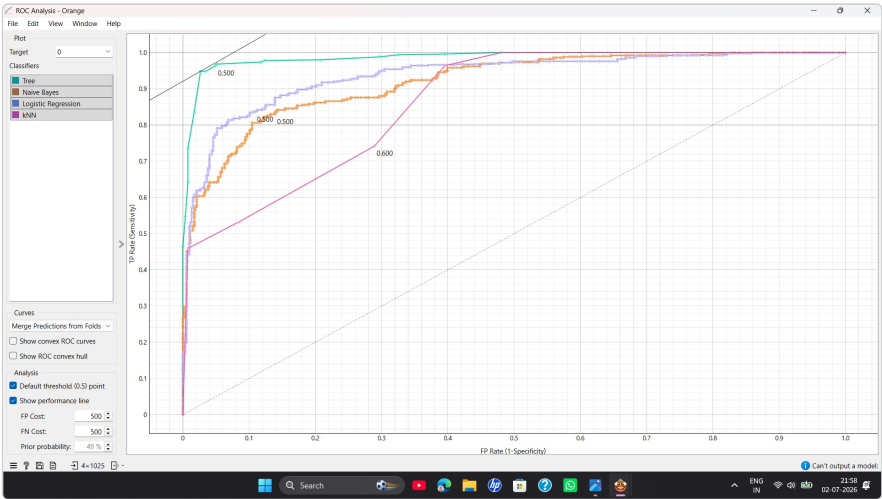
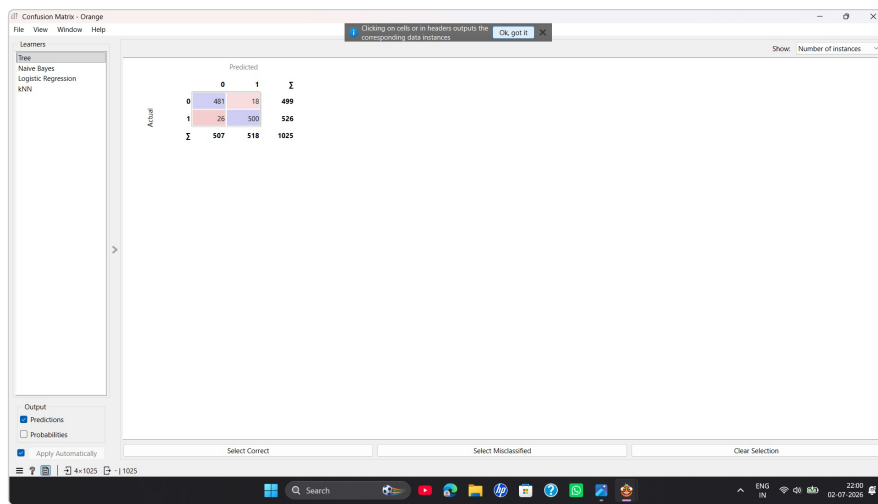


Fig. 10 — ROC curves for all four models (Tree hugs the top-left corner, indicating best performance)

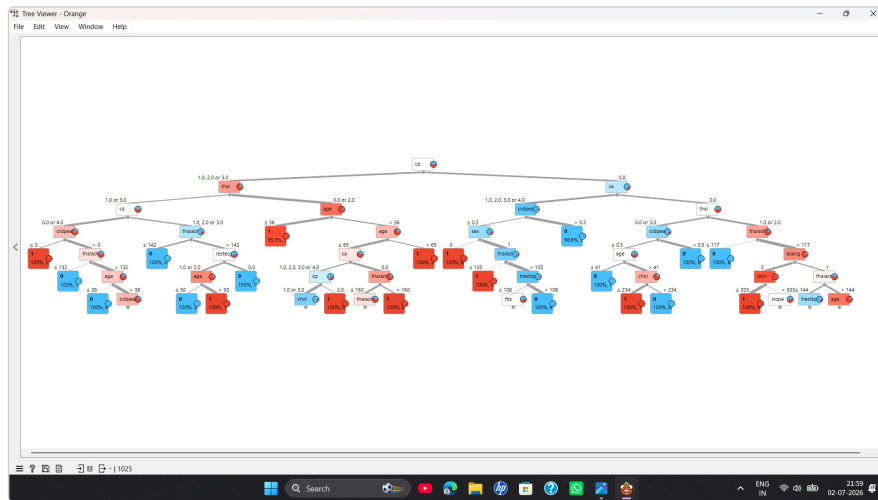
7.3 Confusion Matrix — Tree

	Predicted: No Disease (0)	Predicted: Disease (1)	Total
Actual: No Disease (0)	481	18	499
Actual: Disease (1)	26	500	526
Total	507	518	1025



Only 44 misclassifications out of 1,025 (18 false positives, 26 false negatives) — a strong result for a single decision tree.

7.4 Decision Tree Structure



The first split is on `cp` (chest pain type), followed immediately by `thal` and `ca` — these three dominate the model's predictions, consistent with their clinical significance (vessel blockage and chest pain type are core diagnostic signals for heart disease).

8. Results Summary

($r = -0.39$)

Metric	Value
Best model	Decision Tree
Best AUC	0.986
Accuracy	95.7%
Misclassifications	44 / 1,025
Top predictive features	cp, thal, ca
Strongest correlation	age : thalach

9. Key Takeaways

- **Chest pain type (cp), thal, and ca** are the strongest predictors of heart disease — they form the top splits in the decision tree.
- The **Tree model outperformed every other classifier** (AUC 0.986 vs. 0.937 Logistic Regression, 0.919 Naive Bayes, 0.864 kNN), with the win confirmed as statistically robust.
- **Age is inversely related to max heart rate** ($r = -0.39$) — the strongest pairwise correlation in the dataset, and clinically expected.
- No numeric features show strong multicollinearity, so simpler linear models like Logistic Regression remain reliable and interpretable as a second opinion alongside the Tree.

10. How to Reproduce

1. Install [Orange Data Mining](#) (free).
2. Clone the repository: `git clone https://github.com/<your-username>/heart-disease-orange.git`
3. Open Orange → File → Open → select `workflow/heart_disease.ows` .
4. Run the canvas — all widgets re-execute on `data/heart.csv` .

Built as a hands-on exercise in visual, no-code machine learning with Orange Data Mining.